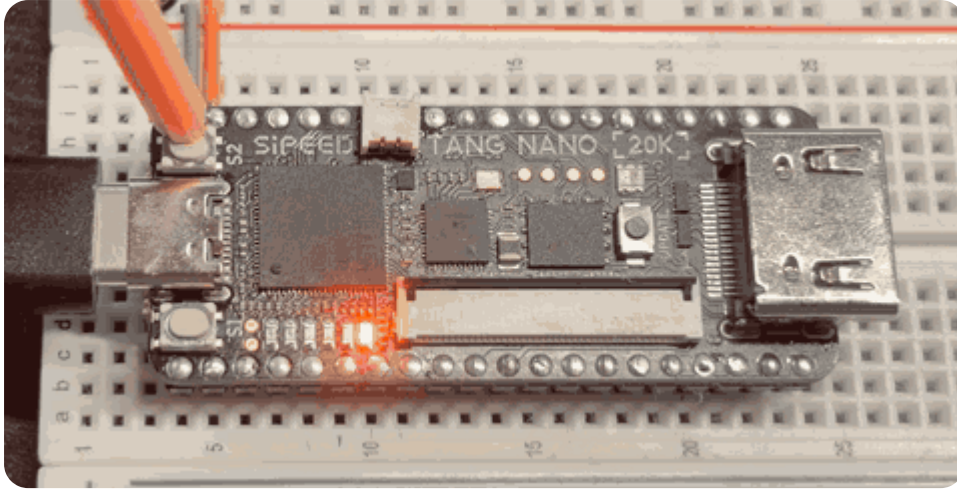


TTA-8 せつめい書 — 8bit の小さい CPU を作って動かそう



このせつめい書では、命令が2つ (READ と WRITE) しかない 8bit CPU 「TTA-8」 を、FPGA ボード Tang Nano 20K の中に作って動かします。小学校 高学年くらいから読めるように書いてあります。

もくじ

章	内容	こんな人に
1. CPU って なんだろう？	CPU・プログラム・2進数	はじめての人
2. TTA-8 の しくみ	READ と WRITE だけで 計算できる ひみつ	CPU の 中を 知りたい人
3. じゅんぴしよう	ボード・ケーブル・道具の 用意	ボードを 買った人
4. 動かしてみよう	LED フラッシャーを ボードに 書き こむ	ボードを 光らせたい人
5. TTA-8 ラボで 中を のぞこう	ブラウザの デバッガの 使いかた	1命令ずつ 見たい人
6. TinyBASIC改 で プログラムを 書こう	BASIC の 文法と サンプル	プログラムを 作りたい 人
7. アセンブリで 書いてみよう	READ / WRITE で 直接 書く	CPU の 気持ちに なりた い人
8. CPU の 設計図を 読もう	Verilog と チューリング完全	もっと 深く 知りたい人
9. AI に こうやって たのもう	こまったときの AI への 聞きかた	だれでも
10. つぎの ぼうけんへ	ブレッドボードで 回路を ふやそう	ぜんぶ 読んだ人
ふろく	こまったとき・用語集・大人の方 へ	保護者・先生

読みかたのヒント ボードがまだなくても、1・2・5・6・7章はブラウザだけでためせます。

このプロジェクトで 使うもの

- **Sipeed Tang Nano 20K** (FPGA ボード)
- **USB ケーブル** (データ通信ができるもの。充電専用は ×)
- **パソコン** (Windows か Mac)
- あれば **ブレッドボード** (ボードを さして 使うと 安定します)

1. CPU って なんだろう？

CPU は「命令を 1つずつ こなす 係」

ゲーム機、スマホ、パソコン、電子レンジ……。その中には **CPU**（シーピーユー）という部品が入っています。

CPU がやることは、じつは とても 単純です。

1. つぎの 命令を 読む
2. その 命令を やる
3. 1 にもどる

これを 1秒間に 何十億回も くりかえしているの、ゲームが動いたり、動画が見られたりするのです。

プログラム とは、CPU にやってほしい 命令を 順番に ならべた ものです。料理の レシピ みたいな ものです。

コンピューターの中は ぜんぶ「数」

CPU は 文字も 絵も 音も、ぜんぶ **数** として あつかいます。しかも その 数は **0 と 1 だけ** で 書かれています。これを **2進数**（にしんすう）といいます。

LED を 6こ ならべて、光っている ところを 1、消えている ところを 0 と すると……

LED	2進数	ふつうの数
____○	000001	1
____○_	000010	2
____○○	000011	3
○_○_○_	101010	42
○○○○○○	111111	63

右はしが 1、そのとなりが 2、つぎが 4、8、16、32 …と、左へ 行くたびに **2倍** になります。101010 は $32 + 8 + 2 = 42$ です。

8bit って？

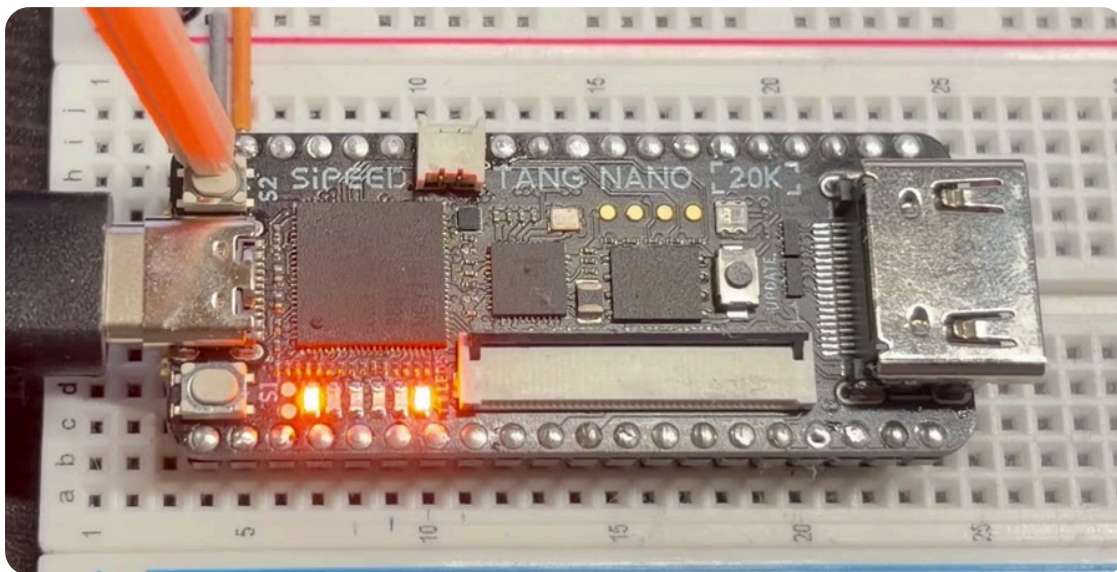
0 か 1 が入る 1マス を **ビット (bit)** といいます。 **8bit** は 8マス なので、 `00000000` (0) から `11111111` (255) まで、 **256 とおりの数** をあらわせます。

このせつめい書で作る CPU「**TTA-8**」は、8bit の数をあつかう CPU です。255 に 1 を たすと、あふれて 0 にもどります (車のメーターが一周するのと同じ)。

FPGA は「回路を書き換えられるチップ」

ふつうの CPU は、工場で作られたあとは中身を変えられません。でも **FPGA (エフピージーエー)** というチップは、**設計図を書きこむと、そのとおりの回路に変身** します。

Tang Nano 20K にのっている FPGA に、TTA-8 の設計図 (Verilog という ことばで書いたもの) を書きこむと、FPGA の中に **ほんものの CPU** ができあがります。



TTA-8 は **1秒に 1000命令** のゆっくりした CPU です。ゆっくりなので、LED の動きを目で追いかけることができます。

2. TTA-8 のしくみ

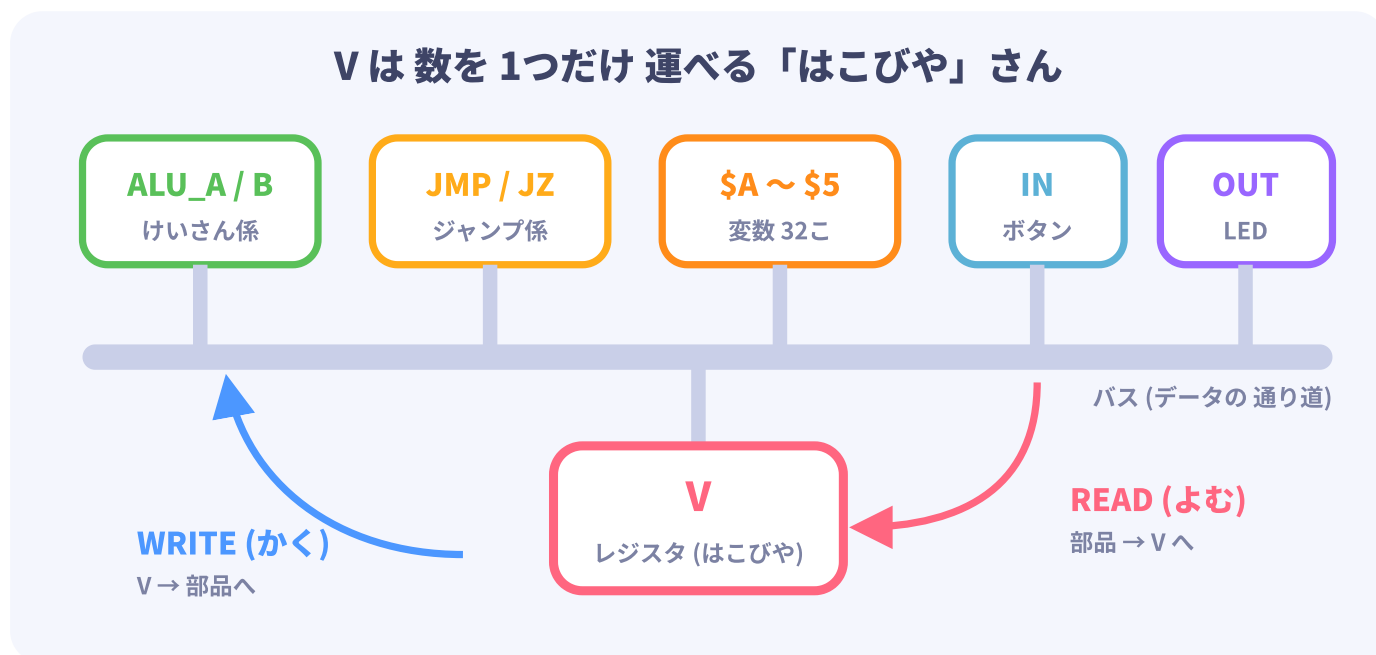
命令は 2 つだけ！

ふつうの CPU には「たし算」「ジャンプ」「比べる」など、何十～何百 種類もの 命令があります。

TTA-8 の 命令は、なんと **2 つだけ** です。

命令	読みかた	すること
READ	よむ	部品から 数を 1 つ 受け取って、V に 入れる
WRITE	かく	V に 入っている 数を、部品に わたす

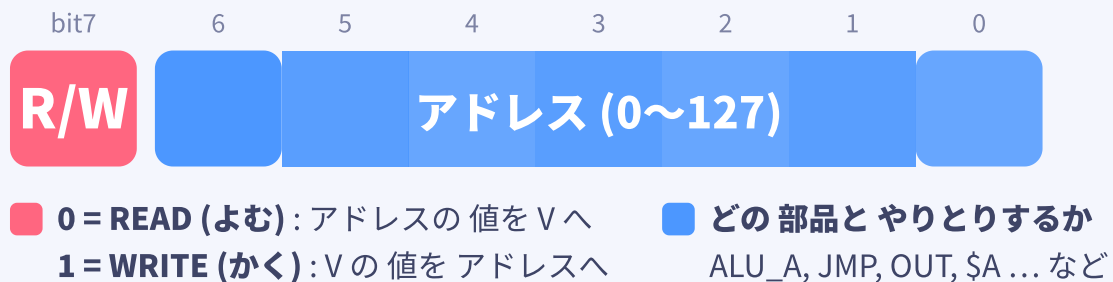
V (ブイ) は、数を 1 つだけ 持てる「はこびや」さんです (レジスタ といいます)。



「え、それだけで 計算 できるの？」 と思いますよね。ひみつは **部品の方が 仕事をする** ことです。けいさん係 (ALU) に 数を わたすと、答えが すぐに 用意 されます。V は それを 取りに 行く だけ。このような CPU を **TTA (Transport Triggered Architecture = 運ぶと 動く しくみ)** といいます。

命令の形

TTA-8 の命令は 1バイト (8bit)



0 1111110 = 0x7E → READ IN (ボタンを よむ)
1 1111111 = 0xFF → WRITE OUT (LED にかく)

1つの命令は 8bit (1バイト) です。いちばん左の 1bit で READ か WRITE かをえらび、のこりの 7bit で「どの部品とやりとりするか」(**アドレス**)をえらびます。

部品とアドレスの一覧（アドレスマップ）

アドレス	名前	READ（よむ）と…	WRITE（かく）と…
0x00	ALU_A	A の数	A に数を入れる
0x01	ALU_B	B の数	B に数を入れる
0x02	ALU_ADD	A + B の 答え	—
0x03	ALU_NAND	A NAND B の 答え	—
0x04	ALU_SUB	A - B の 答え	—
0x10	JMP	—	その 番地へ ジャンプ
0x11	JZ	—	A が 0 の ときだけ ジャンプ
0x12	IMM	命令の つぎに 書いた 数	—
0x20～ 0x3F	\$A～\$Z、\$0～ \$5	変数の 数	変数に 数を入れる
0x7E	IN	ボタン（S1=1、S2=2、両方 =3）	—
0x7F	OUT	—	LED に 出す（1 の ビットが 光 る）

0x が ついた 数は **16進数** です。0x10 は 16、0x7F は 127 の ことです。

ALU（エーエルユー） は Arithmetic Logic Unit（算術論理演算装置）の ことで、CPU の 中で 計算を する 係です。

3 + 5 を計算してみよう

```
READ IMM, 3      ; V に 3 を 入れる
WRITE ALU_A      ; A = 3
READ IMM, 5      ; V に 5 を 入れる
WRITE ALU_B      ; B = 5
READ ALU_ADD     ; V = A + B = 8
WRITE OUT        ; LED に 8 を 出す → 001000
```

V が数を運んでいるだけなのに、ちゃんと足し算ができました。LED には 8 (2進数で 001000) が光ります。

ジャンプも「書くだけ」

CPU は PC (プログラムカウンタ) という番号で「いま何番目の命令か」をおぼえています。ふつうは1つずつ進みますが、

- **JMP** に書くと、PC がその数になります (=ジャンプ)
- **JZ** に書くと、**ALU_A** が 0 のときだけジャンプします

JZ があるので「0 になるまでくりかえす」「ボタンが押されていたらこっちへ」といった**判断**ができます。くわしくは8章で、これだけでどんな計算もできる (**チューリング完全**) ことを説明します。

CPU の中身 まとめ



- PC: いま 何番目の 命令か (8bit)
- V: 数を 運ぶ はこびや (8bit)
- プログラム (ROM): 命令が 256こ まで 入る
- 変数 (RAM): \$A~\$Z と \$0~\$5 の 32こ
- ALU: A と B から たし算・ひき算・NAND を 計算
- IN / OUT: ボタンと LED

これでぜんぶです。CPU の 設計図は、コメントを のぞくと たった 51行 です。

3. じゅんびしよう

用意するもの

もの	ポイント
Sipeed Tang Nano 20K	FPGA ボード。通販で 買えます
USB ケーブル (USB-C)	データ通信ができる もの。充電専用だと ボードが 見えません
パソコン	Windows 10 / 11 か Mac
ブレッドボード (あれば)	ボードを さしておくと 安定します。10章 の ぼうけんにも 使います

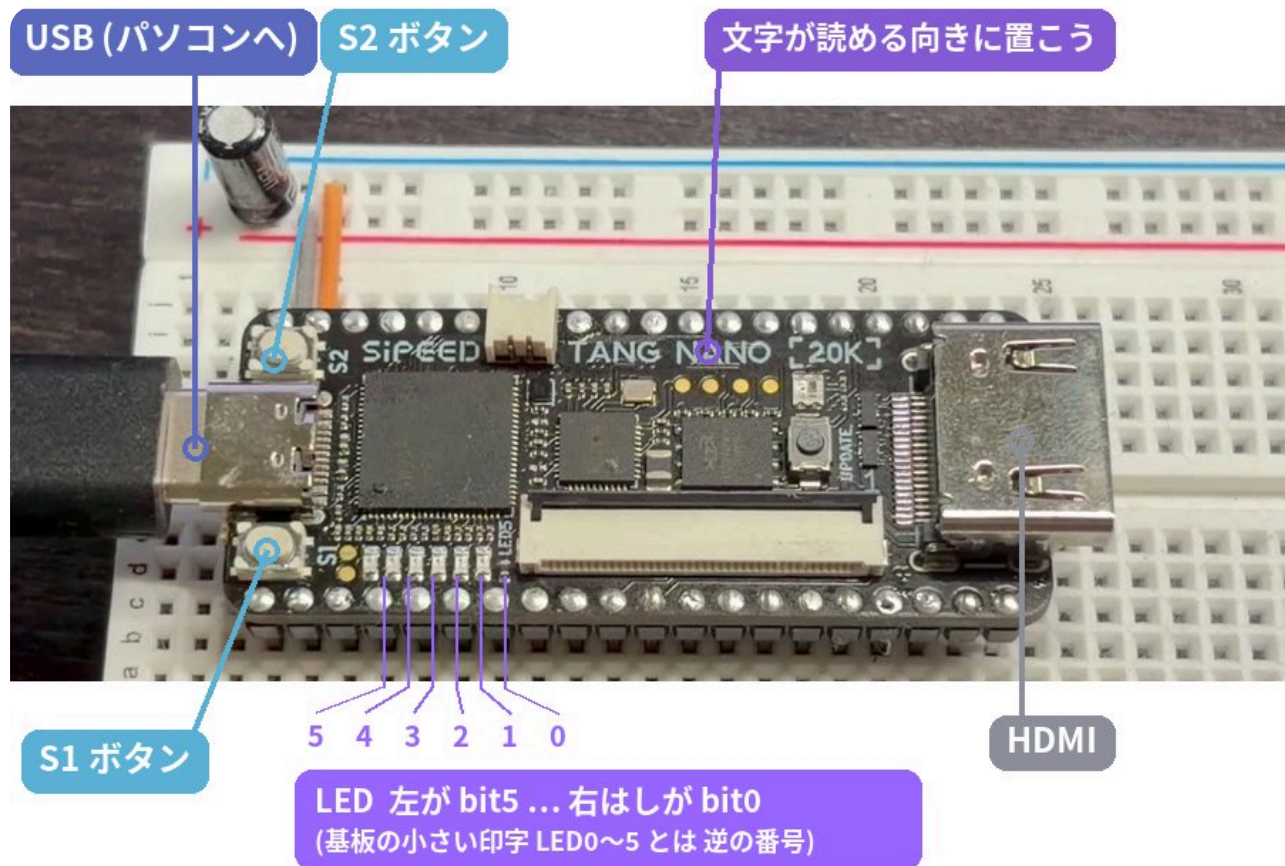
① このプロジェクトを ダウンロードする

1. GitHub の このプロジェクトの ページ (<https://github.com/tjm8874/TTA-8>) を 開く
2. 緑の 「Code」 ボタン → 「Download ZIP」
3. ZIP を 開いて (展開して)、すきな 場所におく

中身は こうなっています。

TTA-8-main/	← ZIP を 開くと この 名前 (TTA-8 に 変えても OK)
└ FPGA_Tool/	← ボードに 書きこむ 道具 (ダブルクリックで 使う)
└ Windows/	
└ Mac/	
└ debugger/	← TTA-8 ラボ (index.html を ブラウザで 開く)
└ hdl/	← CPU の 設計図 (Verilog) と 書きこむ ファイル
└ programs/	← サンプルプログラム
└ docs/	← この せつめい書

② ボードの向きをおぼえよう



「TANG NANO 20K」の文字が読める向き（USBが左）で置きます。この向きで、LEDは**右はしが bit0**、左はしが bit5 です。プログラムで `OUT 1` とすると、右はしの LED が光ります。

基板には小さく LED0~LED5 と印字されていますが、このプロジェクトでは2進数と合わせるために**逆の番号（右はしが0）**を使います。

③ Windows のじゅんび（はじめに1回だけ）

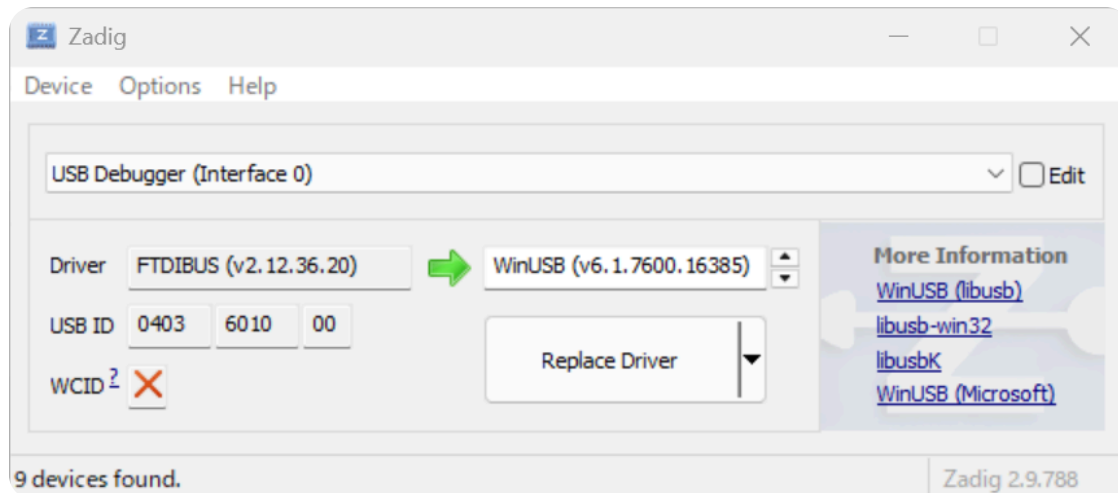
FPGA_Tool\Windows を開きます。

0_setup_driver.bat … USB ドライバの設定

ボードに書きこむ道具（openFPGALoader）がボードと話せるように、**Zadig（ザディグ）**という道具でドライバを入れかえます。

1. ボードを USB でつなぐ
2. `0_setup_driver.bat` をダブルクリック → 何かキーをおすと Zadig が開く
3. 「このアプリが変更を加えることを許可しますか」 → はい

4. メニューの **Options** → **List All Devices** にチェック
5. リストから **USB Debugger (Interface 0)** をえらぶ
6. 下の **USB ID** が **0403 6010 00** になっていれば正解
7. **Interface 1** はえらばない
8. 右がわが **WinUSB** になっているのを確認して **Replace Driver**



1. 「successfully」と出たら完了

リストに出ないときは：「不明な USB デバイス」しか出ないときは、ケーブルが充電専用 かもしれません。べつのケーブルにかえるか、パソコン本体の USB に直接 さして みてください。

1_check_board.bat … ボードが見えるか確認

ダブルクリックして、**model GW2A(R)-18(C)** と出れば OK です。

```
=====
ボードが つながっているか しらべます
=====

--- USB に つながっている もの ---
Bus device vid:pid      probe_type manufacturer serial      product
001 046      0x0403:0x6010 FTDI2232  SIPEED      2025030317  USB Debugger

--- FPGA を さがす ---
Jtag frequency : requested 6.00MHz  -> real 6.00MHz
index 0:
    idcode 0x81b
    manufacturer Gowin
    family GW2A
    model GW2A(R)-18(C)
    irlength 8

*** 「GW2AR-18」 のような 名前が 出ていれば OK! ***
Press any key to continue . . .
```

④ Mac の じゅんび（はじめに 1 回だけ）

Mac ではドライバの設定はいりません。「ターミナル」を開いて、FPGA_Tool/Mac の中でつぎを実行します。

```
bash 0_setup.sh          # openFPGALoader を 入れる（Homebrew が 必要）  
bash 1_check_board.sh    # ボードが 見えるか 確認
```

ファイルをターミナルの画面にドラッグ&ドロップすると、場所を打たなくて済みます。
Homebrew がないときは、9 章のやりかたで AI に聞いてみよう。

4. 動かしてみよう

書きこみかたは 2 しゅるい

ファイル (Windows / Mac)	どうなる？	いつ 使う？
2_write_sram.bat / .sh	おためし。電源を 切ると 消える	何度も ためす とき
3_write_flash.bat / .sh	ずっと。電源を 切っても のこる	完成した とき

さいしょは おためし で やって みましょう。

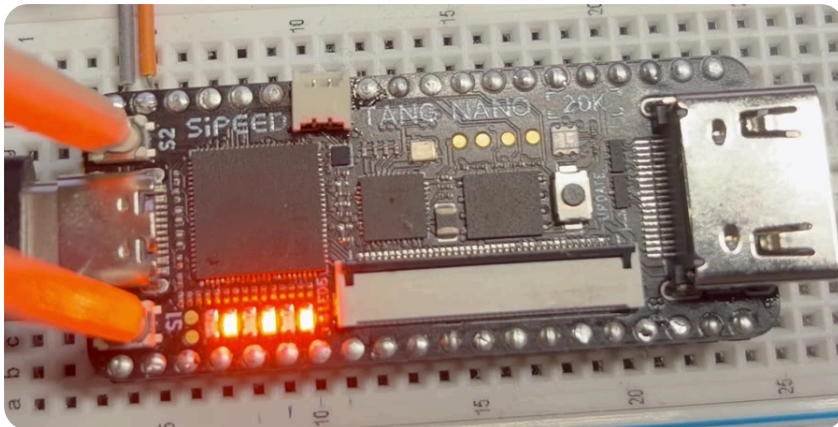
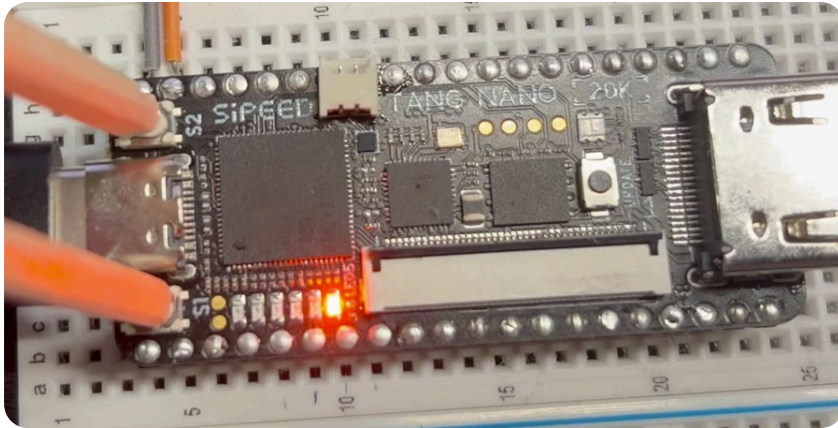
LED フラッシャーを 書きこもう

1. ボードを USB で つなぐ
2. FPGA_Tool\Windows\2_write_sram.bat を ダブルクリック (Mac は `bash 2_write_sram.sh`)
3. 「おわり！ LED を 見てみよう」と 出たら 成功

最初から 入っている `hdl/minimal_cpu.fs` には、LED フラッシャー の プログラムが 入っています。

ボタンを おしてみよう

おす ボタン	LED の 動き (0.5秒ごと)
なし	右はしが 点滅 <code>____○ ⇔ _____</code>
S1	光が 右から 左へ 流れる <code>____○ → ____○_ → … → ○_____</code>
S2	外から 内へ <code>○____○ → _○__○_ → __○○__</code>
S1 と S2	交互に 点滅 <code>○_○_○_ ⇔ _○_○_○</code>



左：なにもおさないとき（右はしが点滅） 右：S1 と S2 を いっしょにおしたとき

電源を切っても動くようにする

`3_write_flash.bat` で書きこむと、USBをぬいてさしなおしても同じプログラムが動きます。少し時間がかかりますが、さいごに **Done** と **おわり！** が出れば OK です。

```
=====
Tang Nano 20K に かきこみ [ずっと]
※電源を きっても 消えませんが。少し 時間が かかります
=====
ファイル : E:\AI\Claude\Minimal_CPU\FPGA_Tool\Windows\...\hdl\minimal_cpu.fs

write to flash
Jtag frequency : requested 6.00MHz  -> real 6.00MHz
Parse file Parse E:\AI\Claude\Minimal_CPU\FPGA_Tool\Windows\...\hdl\minimal_cpu.fs:
Done
DONE
after program flash: displayReadReg 00006020
Memory Erase
Done Final
Security Final
Erase SRAM DONE
Jtag probe limited to 6000000 Hz
Jtag frequency : requested 10.00MHz  -> real 6.00MHz
Detail:
Jedec ID      : 0b
memory type   : 40
memory capacity : 17
Detail:
Jedec ID      : 0b
memory type   : 40
memory capacity : 17
RDSR : 0x00
WIP : 0
WEL : 0
BP : 0
TB : 0
SRWD : 0
flash chip unknown: use basic protection detection
start addr: 00000000, end_addr: 00090000
Erasing: [=====] 100.00%

Done
Writing: [=] 0.13%
Writing: [=====] 22.89%
Writing: [=====] 45.46%
Writing: [=====] 68.62%
Writing: [=====] 91.77%
Writing: [=====] 100.00%

Done

*** おわり！ LED を 見てみよう ***

何か キーを おすと この画面を とじます。
```

黄色い文字で flash chip unknown と出ますが、問題 ありません。

自分のプログラムを書きこむには

ボードの中身はCPU（Verilog）＋プログラム（program.hex）でできています。プログラムを変えたら、CPUといっしょにくみだて直す（ビルドする）ひつようがあります。

プログラムがボードで動くまで

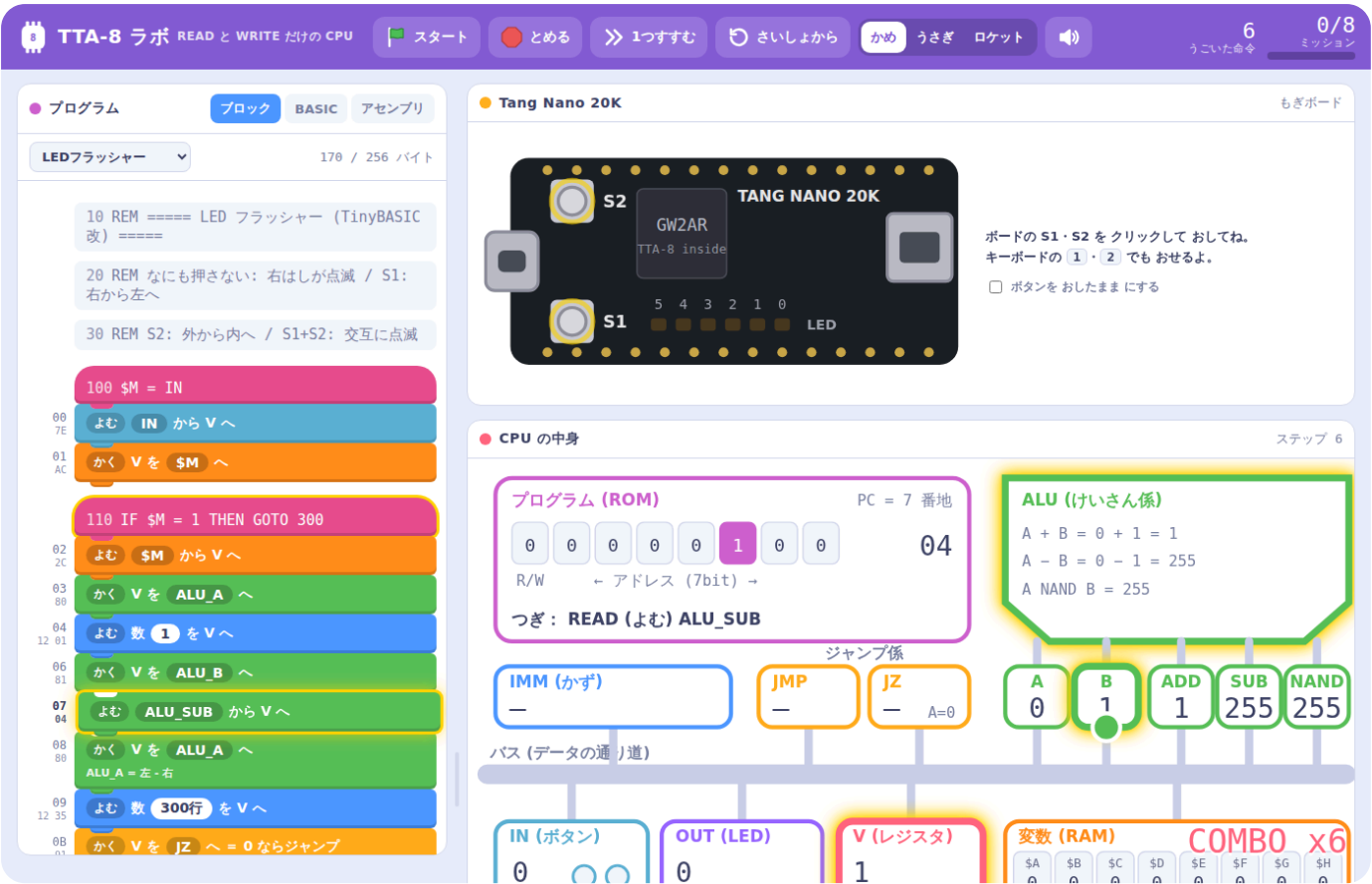


1. 5章の TTA-8 ラボ でプログラムを作り、「ボード用に保存」をおす → `program.hex` が保存される
2. `program.hex` を `4_build.bat` の上にドラッグ&ドロップ (Mac は `bash 4_build.sh program.hex`)
3. 自動で `hdl/program.hex` にコピーされて、ビルドが始まります
4. はじめてのときだけ、ビルド道具 (OSS CAD Suite、約600MB) をダウンロードします
5. 「できた！」と出たら、`2_write_sram.bat` で書きこむ

5. TTA-8 ラボで 中を のぞこう

TTA-8 ラボ は、ブラウザで動く TTA-8 のデバッガです。ボードがなくても、CPU の中で何が起きているかを 1命令ずつ見られます。

debugger/index.html をダブルクリックすると、ブラウザで開きます（インターネットはいりません）。



画面の見かた

場所	見えるもの
左：プログラム	ブロック・BASIC・アセンブリの3つの見かたでプログラムを表示
右上：もぎボード	Tang Nano 20K のまね。S1・S2 をクリックでおせる
右中：CPU の中身	PC・V・ALU・変数・IN・OUT がいまの値といっしょに見える
右下：ミッション	クリアすると ほしがもらえる

動かしかた

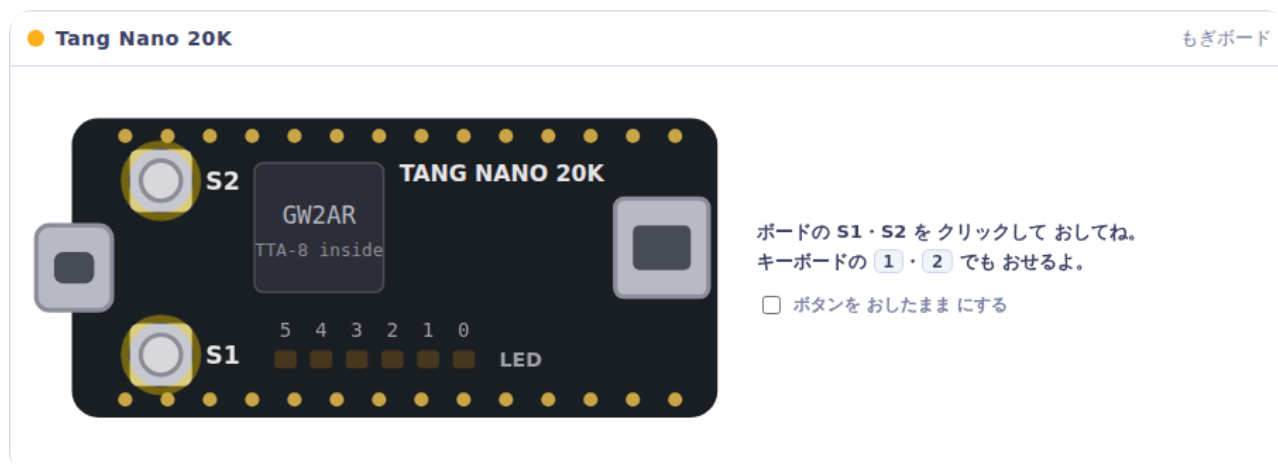
ボタン	キー	すること
スタート（みどりの旗）	スペース	動かす
とめる		止める
1つすすむ	→	1命令だけ 進める
さいしょから		リセット
かめ / うさぎ / ロケット		速さ。ロケット は本物の ボードと 同じ 1秒に 1000命令

1つすすむ を おすと、V が 部品から 部品へ 数を 運ぶ よう すが、バス（データの 通り道）を 走る 玉で 見えます。

- 玉が 部品 → V に 走ったら READ（よむ）
- 玉が V → 部品 に 走ったら WRITE（かく）

キャラクターが「いま 何を したか」を ことばで 教えてくれます。

もぎボード

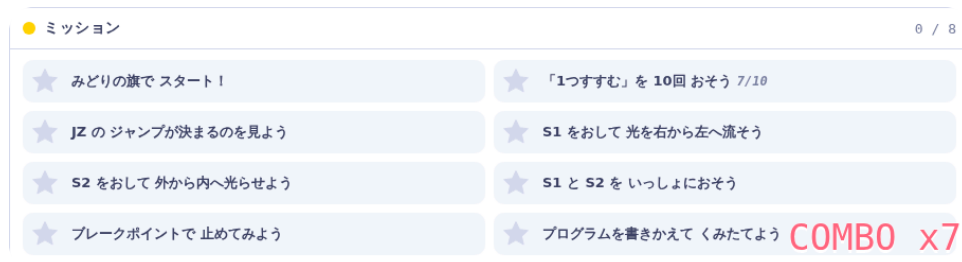


本物の ボードと 同じ 向き・同じ 位置に、S1・S2・LED があります。キーボードの 1・2 でも ボタンを おせます。「ボタンをおしたままにする」に チェックすると、両方をおしたままに できます。

ブレイクポイント

プログラムの 行の 左はしを クリックすると、そこで 止まる しるし（ブレイクポイント）が つきます。ロケットで 走らせて、気になる ところで 止めてみましょう。

ミッション



「1つすすむを 10回 おそう」「JZ の ジャンプが決まるのを見よう」など、8つの ミッションがあります。ぜんぶ クリアできるかな？

プログラムを書きかえる

- BASIC タブ → かきかえる → 書きかえて くみたてる
- アセンブリ タブ → 書きかえて くみたてる
- できたら ボード用に保存 で program.hex を 保存 → 4章 の やりかたで ボードへ

6. TinyBASIC改 で プログラムを 書こう

TinyBASIC改 は、TTA-8 のために作った小さな BASIC です。パソコン（ブラウザ）の上で、TTA-8 の命令（READ / WRITE）に翻訳されます。

はじめてのプログラム

```
10 REM ===== はじめての たし算 =====
20 $A = 3
30 $B = 5
40 $C = $A + $B
50 OUT $C
60 END
```

- 1行ずつ **行番号**（10, 20, 30 …）から 書きます
- **\$A** は **変数**（数を1つしまっておく 箱）
- **OUT** で LED に 出します。8 は 001000 と 光ります
- **REM** の うしろは メモ（プログラムには 関係 ありません）

変数と 数

もの	書きかた	せつめい
変数	\$A ~ \$Z、\$0 ~ \$5	ぜんぶで 32こ。0~255 の 数が 1つ 入る
数	12、0b101010、0x2A	10進・2進・16進。0~255

255 を こえると 0 に もどり、0 より 小さく になると 255 の ほうへ まわります（8bit だから）。

計算

書きかた	意味
<code>\$A + 1</code>	たし算
<code>\$A - 1</code>	ひき算
<code>\$A AND 0b000111</code>	どちらも 1 のビットだけ 1
<code>\$A OR 0b100000</code>	どちらかが 1 のビットは 1
<code>\$A NAND \$B</code>	AND の反対
<code>IN</code>	ボタン (S1=1、S2=2、両方=3)

左から順に計算します（かっこは使えません）。

命令の一覧

命令	例	意味
代入	<code>\$A = \$B + 1</code>	変数に入れる
<code>OUT</code>	<code>OUT 0b000001</code>	LEDに出す（右はしが bit0）
<code>IF ... THEN</code>	<code>IF \$M = 1 THEN GOTO 300</code>	同じなら THEN のあとをやる
	<code>IF IN <> \$M THEN GOTO 100</code>	ちがうなら THEN のあとをやる
<code>GOTO</code>	<code>GOTO 100</code>	その行へとぶ
<code>GOSUB / RETURN</code>	<code>GOSUB 900</code>	その行へ行って、RETURN でもどってくる
<code>WAIT</code>	<code>WAIT 50</code>	50 × 0.01秒 = 0.5秒 まつ
<code>POKE / PEEK</code>	<code>POKE 127, 5</code>	アドレスに直接書く / 読む
<code>END</code>	<code>END</code>	おわり
<code>REM</code>	<code>REM メモ</code>	メモ

： で区切ると、1行にいくつも書けます（例： `$L = 1 : GOSUB 900` ）。

ためしてみよう

LED で 2進数カウンター

```
10 REM ===== LED で 2進数カウンター =====  
20 $C = 0  
30 OUT $C  
40 WAIT 50  
50 $C = $C + 1  
60 GOTO 30
```

0.5秒ごとに1ずつふえて、LEDが2進数で数えます。63のつぎは？考えてからためしてみよう。

かけ算 6 × 7

TTA-8にはかけ算の部品がありません。でも **たし算をくりかえせば** できます。

```
10 REM ===== かけ算 6 x 7 (たし算のくりかえし) =====  
20 $A = 6 : $B = 7 : $C = 0  
30 IF $B = 0 THEN GOTO 70  
40 $C = $C + $A  
50 $B = $B - 1  
60 GOTO 30  
70 OUT $C  
80 END
```

答えの42は 101010。LEDが1つおきに光ります。

LED フラッシャー（ボードに入っているもの）

programs/basic/flasher.bas を TTA-8 ラボの サンプルから えらんで、中を読んでみましょう。IN で ボタンを読んで、IF で 模様を えらんでいます。

できないこと と その かわり

できないこと	かわりの方法
< > で 比べる	<> (ちがう) を 使って くりかえす
かっこ ()	式を 2行に 分ける
GOSUB の 中で GOSUB	もどり先を 1つしか おぼえられない
PRINT (文字を 出す)	画面がないので LED に OUT する

くわしい文法は TinyBASIC改 文法書 を 見てね。

7. アセンブリで書いてみよう

BASIC は便利ですが、CPU がほんとうにやっているのは **READ** と **WRITE** だけです。その命令を直接書くことを **アセンブリ** といいます。TTA-8 ラボの **アセンブリ** タブで書けます。

書きかた

; セミコロンのあとはメモ

```
LABEL: READ 名前          ; 名前の部品から V へ
        WRITE 名前         ; V から 名前の部品へ
        READ IMM, 数       ; 数をそのまま V へ (2バイト 命令)
        READ IMM, LABEL    ; LABEL の番地を V へ (ジャンプ先に使う)
```

READ できる 名前	WRITE できる 名前
ALU_A ALU_B ALU_ADD ALU_NAND ALU_SUB IN IMM 変数	ALU_A ALU_B JMP JZ OUT 変数

変数は \$A ~ \$Z、\$0 ~ \$5 です。

ジャンプのしかた

ジャンプは「行き先の番地を V に入れて、JMP に書く」の2命令です。

```
READ IMM, LOOP    ; V = LOOP の番地
WRITE JMP         ; PC = V → LOOP へジャンプ!
```

もし 0 ならジャンプは、先に ALU_A に調べたい数を入れておいて、JZ に書きます。

```
READ $N
WRITE ALU_A        ; ALU_A = $N
READ IMM, DONE
WRITE JZ           ; ALU_A が 0 なら DONE へ。0 でなければ 次の行へ
```

れんしゅう：5 から 0 まで カウントダウン

```
; 5 から 0 まで 数えて LED に出す
      READ IMM, 5
      WRITE $N          ; $N = 5
LOOP:  READ $N
      WRITE OUT          ; LED に $N を出す
      WRITE ALU_A        ; ALU_A = $N
      READ IMM, DONE
      WRITE JZ           ; ALU_A が 0 なら DONE へ
      READ IMM, 1
      WRITE ALU_B        ; ALU_B = 1
      READ ALU_SUB        ; V = $N - 1
      WRITE $N           ; $N = V
      READ IMM, LOOP
      WRITE JMP          ; LOOP へ もどる
DONE:  READ IMM, DONE
      WRITE JMP          ; ずっと 足踏み
```

TTA-8 ラボに 貼りつけて **くみたてる** → **かめ** の速さでスタート。LED が 5 (000101) → 4 → 3 → 2 → 1 → 0 と 変わって 止まれば 成功です。

ポイント： WRITE OUT のあと、V の中身は 変わらないので、そのまま WRITE ALU_A できます。V を じょうずに 使いまわすのが アセンブリの コツです。

チャレンジ

- 0 から 5 まで **数え上げる** ように 書きかえよう (ヒント：ALU_ADD と、5 を ひいて 0 か 調べる)
- BASIC で 書いた `$C = $A + $B` が どんな アセンブリ になるか、TTA-8 ラボの アセンブリ タブ で 見 てみよう
- ALU_NAND だけで AND を 作れる かな？ (ヒント：NAND を 2 回)

8. CPU の設計図を読もう

TTA-8 の設計図は `hdl/tta8.v` にあります。Verilog（ベリログ）という、回路を書くためのことばで書かれています。コメントをのぞくと **51**行。いっしょに読んでみましょう。

① 部品を用意する

```
reg [7:0] rom [0:255];           // プログラム 256バイト
initial $readmemh("program.hex", rom);

reg [7:0] pc;                    // いま 何番目の 命令か
reg [7:0] v;                    // V : たった1つのレジスタ
reg [7:0] alu_a, alu_b;         // ALU に 入れる数
reg [7:0] ram [0:31];          // 変数 $A-$Z, $0-$5
```

`reg [7:0]` は「8bit の箱」という 意味です。 `program.hex` を ROM に読みこんでいます。

② 命令を分ける

```
wire [7:0] inst = rom[pc];       // いまの 命令
wire      write = inst[7];       // いちばん左の bit : 0=READ 1=WRITE
wire [6:0] addr = inst[6:0];     // のこり 7bit : アドレス
```

2章 の命令の形 そのままです。

③ READ したら 何が読めるか

```
case (addr)
  7'h00:  rdata = alu_a;           // ALU_A
  7'h01:  rdata = alu_b;           // ALU_B
  7'h02:  rdata = alu_a + alu_b;    // ALU_ADD
  7'h03:  rdata = ~(alu_a & alu_b); // ALU_NAND
  7'h04:  rdata = alu_a - alu_b;    // ALU_SUB
  7'h12:  rdata = rom[pc + 8'd1];   // IMM (次の1バイト)
  7'h7E:  rdata = {6'b0, btn};     // IN
  default: rdata = is_ram ? ram[addr[4:0]] : 8'h00;
endcase
```

ALU はここだけです。 + - & ~ を書くだけで、FPGA が計算回路を作ってくれます。

④ 1命令を実行する

```
pc <= pc + 8'd1;           // ふつうは 次へ
if (!write) begin          // ===== READ
  v <= rdata;
  if (addr == 7'h12) pc <= pc + 8'd2; // IMM は 定数を とばす
end else begin              // ===== WRITE
  case (addr)
    7'h00: alu_a <= v;         // ALU_A
    7'h01: alu_b <= v;         // ALU_B
    7'h10: pc <= v;            // JMP
    7'h11: if (alu_a == 0) pc <= v; // JZ (ALU_A が 0 なら)
    7'h7F: led <= v[5:0];      // OUT
    default: if (is_ram) ram[addr[4:0]] <= v;
  endcase
end
```

- READ なら `v <= rdata` (部品 → V)
- WRITE なら `○○ <= v` (V → 部品)
- JMP は「PC に書く」だけ。JZ は「A が 0 なら PC に書く」だけ

CPU の動きが、ほとんどそのまま書いてあるのがわかりますね。

⑤ ボードとつなぐ (top_tangnano20k.v)

hdl/top_tangnano20k.v は、ボードと CPU をつなぐ ファイルです。

- ボードの時計 (27MHz = 1秒に 2700万回) を数えて、**1秒に 1000回** だけ CPU を進める
- ボタンの信号を時計にそろえる
- LED は 0 で光る回路なので、反対 (~led) にしてつなぐ

どのピンにつなぐかは hdl/tangnano20k.cst に書いてあります。

⑥ チューリング完全って？

「どんな計算でも (時間とメモリさえあれば) できる」ことを **チューリング完全** といいます。スマホの CPU もスーパーコンピューターもチューリング完全です。

TTA-8 でできることをならべると……

1. 変数に 1 を **たす** (ALU_ADD)
2. 変数から 1 を **ひく** (ALU_SUB)
3. 変数が **0** かどうかで **ジャンプ** する (JZ)

じつはこの3つがあれば「**カウンター機械 (ミンスキー機械)**」という計算のしくみを作れて、それはチューリング完全だと数学で証明されています。だから TTA-8 も (メモリが 256 バイトまでという点をのぞけば) **チューリング完全** です。

かけ算も、わり算も、ぜんぶ「たす・ひく・0 でジャンプ」のくみあわせで作れます。6章のかけ算 6×7 がその例です。

9. AI に こうやって たのもう

こまったときは、AI（Claude や ChatGPT、Gemini など）に聞いてみましょう。 **じょうずに聞くコツ** は3つです。

1. **何をしようとしているか** を書く
2. **何が起きたか**（エラーの文字や画面）をそのまま見せる
3. **自分のパソコン**（Windows か Mac か）を書く

そのまま使える たのみかた

じゅんびで こまったとき

Tang Nano 20K という FPGA ボードに、openFPGALoader で 書きこもうとしています。
パソコンは Windows 11 です。
1_check_board.bat を 動かしたら、下の 文字が 出ました。どうすれば いいですか？
（ここに 黒い画面の 文字を 貼る）

Mac に Homebrew を 入れたいです。小学生にも わかるように、
ターミナルで 何を 打てば いいか 1つずつ 教えてください。

プログラムを作りたいとき

AI は TinyBASIC改 を 知りません。 **文法書を いっしょに わたす** のが コツです。

下の 文法の「TinyBASIC改」で、LED 6こ を 使った プログラムを 書いてください。
S1 を おしたら LED が 左から 右へ 流れるように したいです。
LED は 右はしが bit0 です。1命令 0.001秒 の CPU です。
（ここに docs/tinybasic.md の 中身を 貼る）

この TinyBASIC改 の プログラムで、LED が 思った とおりに 光りません。
やりたいこと：（ここに 書く）
プログラム：（ここに 貼る）
TTA-8 ラボで 動かすと、（ここに 起きたことを 書く）

しくみを知りたいとき

TTA-8 という、命令が READ と WRITE だけの CPU を勉強しています。
「JZ に書くと ALU_A が 0 のときだけ ジャンプする」の意味を、
小学5年生にわかるように たとえ話で 説明してください。

hdl/tta8.v の この 部分が わかりません。1行ずつ 説明してください。
(ここに Verilog を 貼る)

改造したいとき

TTA-8 の Verilog (下に 貼ります) に、ブザーを 鳴らす 出力を 1つ 足したいです。
アドレス 0x7D に 書くと、ブザーの ON/OFF が 変わるように したいです。
Tang Nano 20K の どの ピンに つなげば いいかも 教えてください。
(ここに hdl/tta8.v と hdl/top_tangnano20k.v を 貼る)

気をつけること

- AI の 答えが **まちがっている** こともあります。TTA-8 ラボで ためして、動くか 確かめよう
- **名前・住所・パスワード** などは 書かないこと
- わからないところは「もっと かんたんに」「たとえ話で」と たのみなおして OK

10. つぎの ぼうけんへ

ここまで 読んだ キミは、

- CPU が **命令を 1つずつ こなす** しくみ
- **READ と WRITE** だけでも 計算・判断・くりかえしが できる こと
- その CPU を **FPGA の 中に 作って 動かす** 方法

を知っています。これは 大人の エンジニア でも しっかり 説明 できる 人が 少ない ことです。

ブレッドボードで 回路を ふやそう

Tang Nano 20K の まわりには、たくさんの **ピン（足）** が ならんでいます。ブレッドボードに させば、ボードの **外がわ**に いろいろな **回路**を つながられます。

つなぐもの	できること	TTA-8 の 改造アイデア
LED を もっと	8こ、16こ の イルミネーション	OUT を 8bit に ふやす
ボタン・スイッチ	入力を ふやす	IN に ビットを 足す
ブザー	音を 鳴らす	新しい アドレスに「音」の 部品を 作る
7セグメント LED	数字を 表示する	変数の中身を 数字で 見る
モーター・サーボ	ものを 動かす	ロボットの 頭脳に する
温度・光センサー	まわりの ようすを 読む	センサーの 値で LED の 色を 変える

TTA-8 に 部品を 足すのは かんたんです。 **新しい アドレス**を 決めて、**READ** か **WRITE** で つなぐだけ。それが TTA（運ぶと 動く しくみ）の いいところ です。

回路をつなぐときは、**USBをぬいてから**。Tang Nano 20K の ピンは **3.3V** です。5V の 部品をつなぐときは 大人と いっしょに 調べよう。

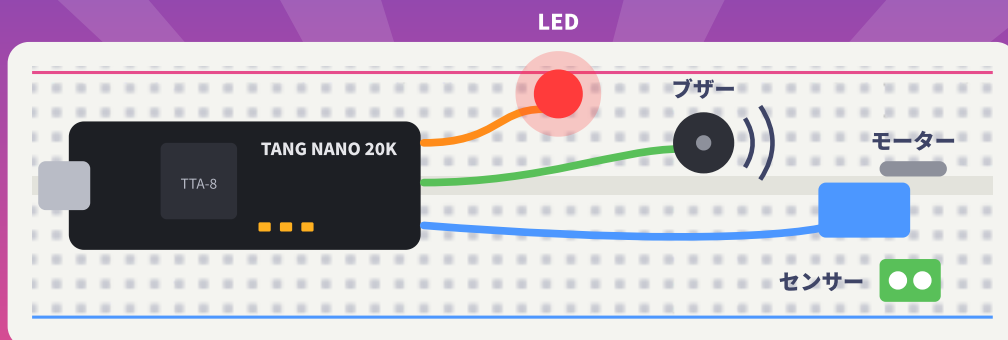
チャレンジ リスト

- ☐ LED フラッシャーに 新しい 模様を 足す
- ☐ 2進数カウンタを 0.1秒ごとに する

- □ かけ算の答えが 63 を こえたら どうなるか 予想して ためす
- □ ALU に「かけ算」の 部品を 足して、アドレス 0x05 で 読めるように する
- □ ブザーをつないで、ボタンでドレミを 鳴らす
- □ 自分で考えた 命令セットの CPU を 作る

おれたちの ぼうけんは これからだ！

ブレッドボードで ボードの外に 回路をつなげよう



TTA-8 のつづきは キミがつくる

ブレッドボードがあれば、このボードの外がわに 色々な 回路を 付けられる。

おれたちの ぼうけん（電子工作）は これからだ！

A. こまったとき

こまったこと	ためすこと
Zadig に「不明な USB デバイス」しか 出ない	充電専用ケーブルかも。データ通信できる ケーブルに かえる／パソコン本体の USB に さす／USB-A ⇄ USB-C ケーブルを ためす
1_check_board で ボードが 見つからない	0_setup_driver を やったか 確認。Interface 0 を WinUSB に したか 確認
書きこめたのに LED が 動かない	もう一度 2_write_sram を ためす。USB を さしなおす
なにも おしてないのに LED が 交互に 点滅	ボタンの 向きの 設定が 逆。hdl/top_tangnano20k.v の BTN_ACTIVE_LOW を 変える（Tang Nano 20K は 0 で 正しい）
光る 向きが 左右 逆	ボードの 向きを 確認（「TANG NANO 20K」が 読める 向き）
4_build の ダウンロードが 止まる	インターネット接続を 確認して もう一度。社内・学校の ネットワークでは 止められる ことも
TTA-8 ラボで「くみたて失敗」	下の 赤い 文字を 読む。わからなければ 9章 の やりかたで AI に 聞く
Windows で 文字が 化ける	Windows 10 の 古い版では 起きることが あります。Windows Update を してみてください

B. 用語集

ことば	意味
CPU	命令を 1 つずつ こなす 部品
FPGA	設計図を 書きこむと その回路になる チップ
ビット (bit)	0 か 1 が 入る 1 マス
バイト	8 ビット。0 ~ 255
2 進数 / 16 進数	0 と 1 だけの 数 / 0 ~ 9 と A ~ F の 数 (0x を つける)
レジスタ	CPU の 中の 小さな 箱。TTA-8 では V
PC (プログラムカウンタ)	いま 何番目の 命令かを おぼえる 箱
ALU	計算を する 係
アドレス	部品や メモリの 番号
ROM / RAM	プログラムを しまう ところ / 変数を しまう ところ
バス	データの 通り道
アセンブリ	CPU の 命令を そのまま 書く ことば
コンパイル	BASIC などを 機械語に 翻訳する こと
Verilog	回路を 書く ことば
ビルド	Verilog と プログラムから 書きこみ用 ファイルを 作る こと
チューリング完全	どんな 計算でも できる こと

C. 大人の方へ（保護者・先生・開発者）

構成

パス	内容
hdl/tta8.v	CPU 本体（コメント除き 51 行）
hdl/top_tangnano20k.v	ボード用トップ（27MHz → 1kHz ステップ、ボタン同期、LED 反転）
hdl/tangnano20k.cst	ピン割り当て（LED は bit0 = pin 20 … bit5 = pin 15 に反転）
hdl/program.hex	ROM 初期値（256 行の 16 進）
hdl/minimal_cpu.fs	ビルド済みビットストリーム（LED フラッシャー）
tools/	tta8asm.js（アセンブラ）、tbasic.js（TinyBASIC改 コンパイラ）、tta8sim.js（エミュレータ）、test.js
debugger/	TTA-8 ラボ（src/lab.html を build.py で index.html に 1 ファイル化）
FPGA_Tool/	書き込み・ビルド用スクリプト、openFPGALoader（Windows 版同梱）、Zadig
sim/tb_flasher.v	iverilog 用テストベンチ
docs/architecture.md / docs/tinybasic.md	仕様書・文法書

コマンドラインでのビルド

OSS CAD Suite（yosys / nextpnr-himbaechel / apicula）を使います。FPGA_Tool/*/4_build と同じ内容です。

```
cd hdl
yosys -p "scratchpad -set abc9.xaiger 1; read_verilog top_tangnano20k.v tta8.v; synth_gowin -top top
nextpnr-himbaechel --json build/top.json --write build/pnr.json \
    --device GW2AR-LV18QN88C8/I7 --vopt family=GW2A-18C --vopt cst=tangnano20k.cst
gowin_pack -d GW2A-18C -o minimal_cpu.fs build/pnr.json
openFPGALoader -b tangnano20k minimal_cpu.fs      # SRAM
openFPGALoader -b tangnano20k -f minimal_cpu.fs    # Flash
```

scratchpad -set abc9.xaiger 1 は、Windows 版 OSS CAD Suite (2026-09-29) で ABC9 が XAIGER2 の読み込み時にアサーションで落ちる問題の回避策です (旧 XAIGER 経路を使う)。

使用リソース：LUT4 約 700 / 20,736 (3%)、DFF 110。ROM は LUT で構成しています。

プログラムのコンパイル (Node.js)

```
node tools/tbasic.js programs/basic/flasher.bas  # → .asm / .hex / .lst
node tools/tta8asm.js programs/flasher.asm      # → .hex / .lst
node tools/test.js                             # コンパイラのテスト
```

Gowin 公式ツールを使う場合

Gowin EDA (Education 版) でも hdl/*.v と hdl/tangnano20k.cst からビルドできます。ただし Zadig で WinUSB に置き換えた後は Gowin Programmer から認識しにくくなります。戻すときはデバイスマネージャーで該当デバイスのドライバを削除して再接続してください。

動作確認

2026-09-29、実機 (Tang Nano 20K) で SRAM・Flash 書き込みとも、なし/S1/S2/S1+S2 の全モードの動作を確認しています。